Computers & Security 151 (2025) 104356

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

PatchView: Multi-modality detection of security patches

Nitzan Farhi [a],*, Noam Koenigstein [b], Yuval Shavitt [a]

[a] School of Electrical Engineering, Tel-Aviv University, Tel-Aviv, Israel
[b] Department of Industrial Engineering, Tel-Aviv University, Tel-Aviv, Israel

ARTICLE INFO

Keywords:
CVE
Machine learning
Git
GitHub
Behavioral data
LSTM
Conv1D

ABSTRACT

Patching software become overwhelming for system administrators due to the large amounts of patch releases. Administrator should prioritize security patches to reduce the exposure to attacks, and can use for this task the Common Vulnerabilities and Exposures (CVE) system, which catalogs known security vulnerabilities in publicly released software or firmware. However, some developers choose to omit CVE publication and merely update their repositories, keeping the vulnerabilities undisclosed. Such actions leave users uninformed and potentially at risk. To this end, we present PatchView, an innovative multi-modal system tailored for the classification of commits as security patches. The system draws upon three unique data modalities associated with a commit: (1) Time-series representation of developer behavioral data within the Git repository, (2) Commit messages, and (3) The code patches. PatchView merges three single-modality sub-models, each adept at interpreting data from its designated source. A distinguishing feature of this solution is its ability to elucidate its predictions by examining the outputs of each sub-model, underscoring its interpretability. Notably, this research pioneers a language-agnostic methodology for security patch classification. Our evaluations indicate that the proposed solution can reveal concealed security patches with an accuracy of 94.52% and F1-scoreof 95.12%. The code for this paper will be made publicly available on GitHub: https://github.com/nitzanfarhi/PatchView.

1. Introduction

The vast majority of software today is developed collaboratively, where developers work separately on different parts of the software, and then merge their code into the rest of the project. To allow this collaboration, source control tools, such as Git (Chacon and Straub, 2014), were developed. Git is a standard protocol that supports file versioning, which is extensively used by programmers to collaborate on software development. A repository of files is maintained by Git, allowing operations such as updating files (push/commit), merging file versions (merge), and splitting a project into two branches (fork).

A hosting service such as GitHub[1] allows users to maintain and share a repository and also allows tracking issues such as bug reports, feature suggestions, fork repositories, maintain merge requests, etc. This makes GitHub a software ecosystem, where interaction among developers reveals patterns of behavior (Horawalavithana et al., 2019).

Software is naturally prone to errors, some of which can be exploited by malevolent entities for malicious purposes. These errors are commonly referred to as *vulnerabilities* and are typically addressed by a software update known as a *security patch*. Vulnerabilities can be categorized into two main types: *0-day*, which refers to vulnerabilities that are unknown to the public and lack a corresponding security patch, and *N-day*, which refers to vulnerabilities for which a security patch has already been released.

The Common Vulnerabilities and Exposures[2] (CVE) is a glossary of publicly disclosed vulnerabilities. Each entry indicates the date of the vulnerability's disclosure, its description, references, and associated metrics, including the complexity of the vulnerability and the necessity of user interaction. For analytical purposes, vulnerabilities are carefully examined, and the Common Vulnerability Scoring System (CVSS) is used to measure the threat level. This CVE score often dictates the urgency and prioritization of vulnerability patching.

Upon identification and rectification of a vulnerability, it is conventional for developers to issue a CVE as a cautionary measure, intending to prompt the user community to install the requisite security patch. However, some developers opt to conceal vulnerabilities, choosing instead to merely amend their repository without issuing a CVE. Various motivations might drive this behavior, including providing early access to the patch for select users, adhering to the "security by obscurity" doctrine, or avoiding public embarrassment.

* Corresponding author.
E-mail address: nitzanfarhi@mail.tau.ac.il (N. Farhi).

[1] https://github.com/
[2] https://cve.mitre.org/

https://doi.org/10.1016/j.cose.2025.104356
Received 13 July 2024; Received in revised form 24 November 2024; Accepted 22 January 2025
Available online 30 January 2025
0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

The main risk stemming from not disclosing vulnerabilities is keeping the users uninformed regarding potential compromises in their software necessitating an update. Conversely, malicious actors might discern alterations in the code, deduce the vulnerability from the newly introduced patch, and subsequently exploit it.

Recent works (Sun et al., 2020; Wang et al., 2021b) have shown the ability to analyze source code patches and commit messages using natural language processing models. However, these approaches exhibit significant limitations that hinder their applicability to real-world scenarios. Specifically, these models:

- Support only a single programming language, restricting their ability to generalize to multi-language codebases.
- Are trained on small datasets confined to specific repositories or languages, making them less representative of diverse real-world settings.
- Certain approaches rely solely on commit messages, which may vary in quality and language, thereby reducing the reliability of the resulting predictions. Other methodologies incorporate source code patches alongside commit messages, however, the integration of these two modalities is often executed in a simplistic manner.

Additionally, these methods lack the ability to incorporate developer behavioral data, which could reveal patterns indicative of concealed security patches. As a result, they are less effective for detecting vulnerabilities. For example, commit messages are sometimes incomplete, ambiguous, or written in natural languages other than those used to train the model, in these cases, the models will not be able to detect the security patches.

These limitations underscore the need for a more robust, language-agnostic, and behavior-informed approach to address the challenges of security patch detection effectively. Additional comparisons with these recent works are provided in Section 2, where we demonstrate superior performance due to our model's ability to integrate developer behavioral data and utilize a broader dataset that encompasses multiple programming languages.

In this paper, we propose PatchView - a language-oblivious approach for classifying commits as security patches. The solution for this problem requires different sources, therefore, our system trains three deep learning sub-models, each designed to harness a different data modality, to detect concealed CVEs. The three sub-models are then assembled together to create a multi-modal composed approach that excels in classifying commits using different data sources. The data modalities employed in this research are: (1) developer behavioral data, (2) commit messages in natural language, and (3) code patches in different programming languages. This is the first time a behavioral model is suggested for this problem. Our evaluation shows that the final composed model achieves state-of-the-art results with an accuracy of 94.52% and an F1-score of 95.12% in detecting undisclosed security patches.

In our final multi-modal solution, end-to-end fine-tuning further improved accuracy, highlighting PatchView's efficacy in synthesizing diverse information sources. For reproducibility, we have made our data extraction code and models publicly available on GitHub[3] and the dataset on Kaggle.[4] Our extensive dataset, covering more repositories than any previous public dataset, can serve as a valuable reference for detecting concealed security patches.

Our key contributions in the paper are as follows:

- PatchView- a novel multi-modality model designed to identify concealed security patches in commits. Its main advantages are:
  - Oblivious to the programming language.
  - Oblivious to the natural language used in the commit message.
- A novel behavior model for this problem. This is the first time behavioral model is suggested for this problem.
- Novel methodologies for encoding code patches are introduced based on a composition of sub-models, each designed to handle a different data modality.
- A large comprehensive and diverse dataset comprising of a balanced mix of labeled security patches and non-security patches.
- Emphasizing model explainability, we provide insights into the factors that most influence the predictions, fostering trust, and facilitating effective validation of results.
- We improved the prior best results for accuracy and F1-Score by about 2% and 6% points, respectively.

Overall, these contributions advance the state-of-the-art in security patch detection, providing valuable tools and knowledge for improving software security and mitigating potential vulnerabilities.

2. Related work

In this section, we present an overview of previous studies of security patch detection using the CVE Program. We begin by discussing papers that focus on analyzing code patches to generate suitable commit messages. Subsequently, we discuss papers that examine the meta-data of a repository for various purposes. Also, we review studies focused on detecting security patches from Git commits. Lastly, we examine research within the cybersecurity domain that utilizes comparable multimodal approaches.

Code analysis. Jung (2021) suggested a system that feeds code patches of commits together with relevant commit messages to fine-tune a RoBERTa model (Liu et al., 2019), a variant of the BERT (Devlin et al., 2018) model with modifications in its training approach and data preprocessing. Hoang et al. (2020) suggested generating a commit message by using Hierarchical Attention Network (HAN) models, one model for added code lines and another model for deleted lines. The two models' outputs were merged into a fully connected layer to predict words and generate a commit message.

Metadata analysis. Only a few studies considered analyzing the meta-data from Git repositories. Pickerill et al. (2020) used meta-data of Git logs for detecting repositories that are "engineered", namely repositories that are not personal nor inactive. Specifically, they crafted explicit features such as integration frequency, commit frequency, integrator frequency, committer frequency, and merge frequency, and transformed them into a time series, which was used to detect whether a repository is "engineered" or not. Coelho et al. (2018) used GitHub's API to extract the number of forks, open issues, closed issues, commits, and max days without commits to detect whether a project is under maintenance or unmaintained.

VCCFinder (Perl et al., 2015) was the first to focus on identifying concealed security vulnerabilities in code commits. Their work was restricted to *C/C++* repositories, relied on limited meta-data, including fork count", star count, number of commits, and programming language (C or C++). Notably, they did not incorporate historical Git meta-data. Additionally, they employed a Support Vector Machine (SVM) model (Steinwart and Christmann, 2008) to detect concealed security patches in commits.

While the aforementioned works utilized Git behavioral meta-data, they were limited in the types of meta-data employed or focused on a different task altogether. This work is the first to utilize a time series of developer behavioral data, extracted from repositories' meta-data changes, for the task of security patch classification.

Security Patch Classification. PatchDB (Wang et al., 2021a) is a collection of security patches for *C/C++* projects. It shows the changes made to the code and the related commit messages. This dataset was built by pulling out security patches using the CVE Program, a method

[3] https://github.com/nitzanfarhi/PatchView
[4] https://www.kaggle.com/datasets/nitzanfarhi/patchview

we also use for our data collection. In PatchRNN (Wang et al., 2021b), LSTM neural networks were employed to figure out which commits are security updates. They tokenized the commit messages and code patches, which were then input into the LSTM model, achieving a final prediction accuracy of 83.57%.

Le et al. (2021) proposed a method to identify the vulnerability-contributing commit, which are the specific code patches that introduce vulnerabilities into the software codebase, for instance, by utilizing the 'git blame' command. Tokenization was conducted on vulnerability-contributing commits within 246 *Java* repositories, and an attention-based GRU model was utilized to classify the severity of the commit.

PatchScout (Tan et al., 2021) used a correlations-based system to detect the location of a vulnerability using the security patch description given in the NVD database. Their system utilized features such as shared words between the code and the NVD description, to detect the location of the vulnerability in the code.

Zhou et al. (2021) selected 4 open-source *C/C++* projects and filtered commits by using known keywords that describe a security patch. Then, security researchers manually labeled the commits as either commits that *introduces* a vulnerability, commits that *closes* a vulnerability, or commits that are *unrelated* to vulnerabilities. NLP methods were used to create a deep learning model based on LSTM and Convolutional Neural network, which led to an accuracy of 90%.

Li and Paxson (2017) also used the CVE Program to extract security patches from Git repositories. This was done to analyze and gain insights into the security patches. The authors investigated the number of changes done to the code base, the maturity of the bugs, and the number of bugs that were solved by the commits.

Sun et al. (2020) extracted security patch commits from the CVE Program and verified that the commits are security patches by manually inspecting 200 of them. Then, the authors extracted the commit message only from GitHub (without using other features such as added lines, removed lines, or date). Repositories that contained C/C++ code were selected, and non-security patches were acquired by taking all other commits from the same repositories. The NLP model that was used to detect the security patches was a hierarchical attention network with GRUs (Cho et al., 2014) as its layers. The authors reached an accuracy of 92%. Zuo et al. (2023) employed a similar method, using a transformer model to identify security patches from commit messages.

The VulCurator system (Nguyen et al., 2022) adopted a similar approach to classify commits as security patches. They utilized the SAP dataset, which consists of 220 Java projects containing labeled security patch commits. Additionally, the authors incorporated the TensorFlow open-source project (Abadi et al., 2015), which also provided labeled commits for their analysis. Similarly to our study, this paper also leverages different data sources to classify commits. Specifically, they used: code patches, commit messages, and issue reports. However, different from (Nguyen et al., 2022), our work does not include issue reports as a data source due to limited availability in most repositories. By contrast, we focused on behavioral data, which is present in all repositories and provides valuable insights into commit classification.

As previously discussed, the aforementioned studies were restricted in the data modalities they employed. In contrast, this study utilized all available modalities. In Section 6, we demonstrate the advantages of the multi-modal approach over single-modality alternatives. Furthermore, prior studies were limited to a small selection of code repositories, while this work employed all repositories available in the NVD database.

Multi-modal models. Multi-modal models, which integrate various data sources, have been increasingly utilized across several areas of cybersecurity. For example, in a study conducted by Lee et al. (2024), the detection of phishing websites was achieved by combining modalities from both the text and screenshots of websites. The modalities were subsequently analyzed using a prompting framework to assess whether the data originated from a phishing site. This method is more straightforward, as it does not require model training.

In the domain of cyber bullying detection, a more intricate multi-modal approach was adopted by Pericherla and Egambaram (2021). They employed the RoBERTa language model for extracting text from tweets and the Xception CNN for analyzing images. The resulting embeddings from both models were concatenated, after which a LightGBM decision model was applied to assess whether a tweet contained bullying content. This advanced fusion of text and image processing demonstrates the expanding role of multimodal models in addressing cybersecurity challenges.

A similar approach was proposed by Moraliyage et al. (2022), where the classification of onion services was performed using both website screenshots and text data. Each modality was processed separately, with CNNs handling images and LSTMs managing text, and an additive method was used to combine the embeddings from both models. The final decision was then made based on the fused embedding.

In this study, a comparable methodology is implemented, where different models handle distinct modalities, and the resulting embeddings are combined for further training. However, a key distinction lies in the end-to-end training phase employed in our multimodal model, which follows the initial individual model training. This methodology is further elaborated in Section 4.1.

3. Multi-modal data for security patches

In this section, we detail the process of collecting data for training both the individual sub-models and the integrated model. This dataset, alongside the code for our proposed model, is publicly accessible in our Git repository. The availability of this dataset represents a significant contribution, considering the scarcity of public datasets for CVE detection.

3.1. Dataset construction

Our primary source for gathering a dataset of security patches was the CVE Program (CVE), the largest database of reported vulnerabilities. Each CVE record includes a unique vulnerability identifier formatted as CVE-yyyy-xxxxx, where 'yyyy' represents the publication year and 'xxxxx' is a sequential ID for that year. This identifier is crucial for associating a CVE with related data in Git repositories.

The CVE Program releases all CVEs in a comma-separated values (CSV) format, containing each CVE's description and supplementary links. We utilized these links as follows: the 'additional links' field was parsed to isolate individual URLs, and any link directing to a GitHub commit was used to compile a tuple ⟨CVE-ID, commit-URL⟩. The commit URL, comprising the commit hash, repository name, and owner, uniquely identifies a commit on GitHub.

We collected CVEs linked to GitHub commits containing security patches from 2015 to 2021. Additionally, we gathered non-security patch commits from the same repositories, ensuring a diverse dataset for identifying both security and non-security patches. This approach could be further refined by leveraging any information provided in CVE descriptions to trace the corresponding commit (Sabetta et al., 2024).

Our collection encompasses approximately 27,130 commits from 1679 repositories, with these commits comprising 3,692,098 lines added and 2,749,243 lines deleted. The dataset includes 27,130 overall events and 8,707,230 words within the commit messages. To ensure a diverse dataset conducive to identifying both security and non-security patches, we also collected non-security patches from the same GitHub repositories as the security patches.

Each Git commit is composed of multiple modalities:

- Behavioral data — a time series composed of developers' interactions with the repository as will be explained below.
- The commit message — a message that describe the content of the code change.

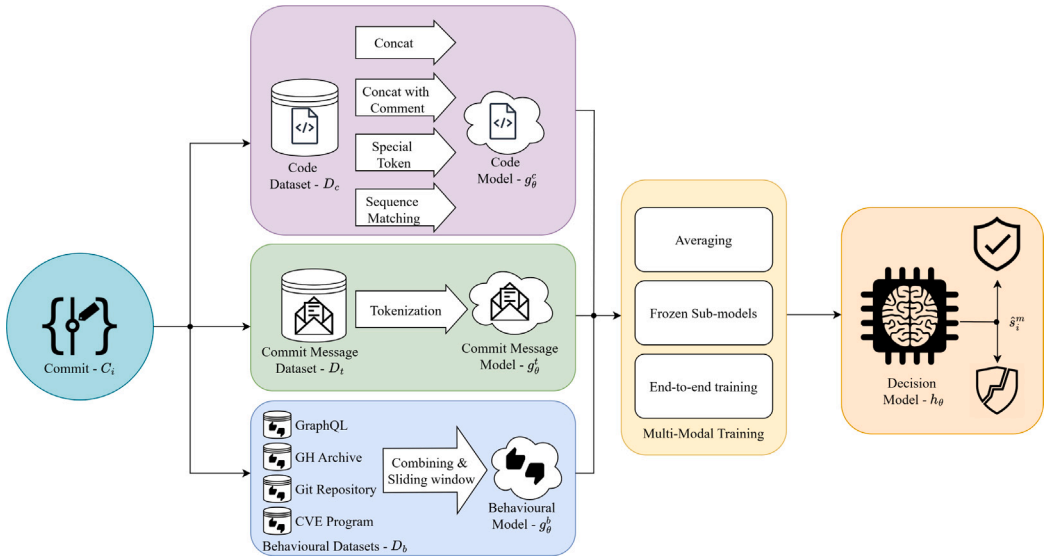

Fig. 1. An overview of PatchView: the streamlined process involving data extraction from multiple sources using commit hashes. The extracted data trains a matching model, followed by a composed decision model that combines sub-models and can be trained further, to make informed decisions.

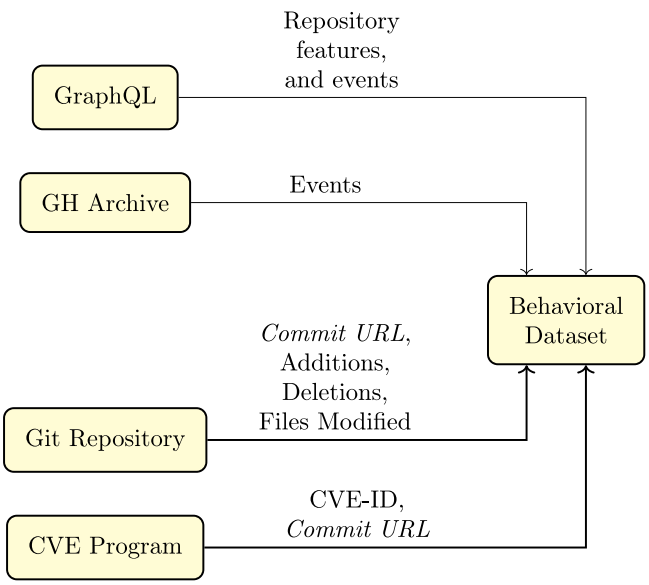

Fig. 2. Behavioral data collection process.

- The code — For each file in the repository, a list of code lines added and deleted.

From each commit identifier we obtained the data from all three modalities, as shown in Fig. 1. In what follows, we describe in detail how the data was obtained and processed.

3.2. Behavioral dataset

The behavioral dataset aggregates information about developers' interactions with the repository from multiple sources (Farhi et al., 2023), including GraphQL, GH Archive, the Git repository, and the CVE program for labeling, as illustrated in Fig. 2. These sources provide event features, which describe individual events, some sources might contain also repository features, which describe the entire repository. Following preprocessing, the data from these sources are consolidated into a unified dataset, resulting in a total of 401 behavioral features.

GraphQL. The GraphQL[5] API facilitates the iteration over the entire repository's history and the collection of all events. It is important to note that only data from the "master", "main", or other active branches are collected, as inactive branches are not tracked.

The detailed collected data fields are presented in Table 1, which contains descriptions of each feature, its data type, and whether it is an event or a repository feature. Events might lack additional data and therefore may not contain a specific data type. The various preprocessing methods applied to each feature will be discussed in Section 3.5.

GH Archive. Furthermore, we extracted data from "GH Archive",[6] a repository of GitHub events. This database contains events that cannot be obtained via GraphQL because they are recorded in real-time, and it does not retain data from deleted branches. We included all repositories linked to CVEs and collected events from "GH Archive". The extracted data fields from GH Archive are listed in Table 2. Features that overlap with GraphQL were merged. All features from this data source are events without data type.

Furthermore, data were extracted from "GH Archive",[7] a repository of GitHub events. This database includes events that cannot be obtained via GraphQL, as they are recorded in real-time and do not retain data from deleted branches. We incorporated all repositories linked to CVEs and collected events from "GH Archive". The data fields extracted from GH Archive are detailed in Table 2, with overlapping features from GraphQL being consolidated. Notably, all features sourced from GH Archive are events and lack specific data types.

Git Repository. Finally, we acquired the mapping between a commit ID and its timestamp, the number of additions, deletions, and the number of files that were changed. Commit is labeled as either *positive* - namely, a security patch or *negative* - non-security patch. Since all aforementioned data is also dated, we overall have a dataset consisting of temporal time series of behavioral events with their date, time, and label.

3.3. Commit message

The process of acquiring commit messages is straightforward, as they are readily available as a field within each commit entry. Commit

[5] https://graphql.github.io/
[6] https://www.gharchive.org/
[7] https://www.gharchive.org/

Table 1
Features collected from GraphQL.

Feature	Data type	Description	Feature type
Additions	Integer	Lines added to the repository	Event
Comment		Comment added to a commit	Event
createdAt	Integer	Year repository created	Repository feature
Deletions	Integer	Lines deleted from the repository	Event
diskUsage	Integer	Repository disk usage (KB)	Repository feature
Fork		Repository forked	Event
hasIssuesEnabled	Boolean	Issues enabled	Repository feature
hasWikiEnabled	Boolean	Wiki enabled	Repository feature
isCompany	Boolean	Owner is a company	Repository feature
isEmployee	Boolean	Owner is an employee	Repository feature
isInOrganization	Boolean	Owner in an organization	Repository feature
isMirror	Boolean	Repository is a mirror	Repository feature
isPolicyEnabled	Boolean	Security policy enabled	Repository feature
isSponsored	Boolean	Owner is sponsored	Repository feature
isVerified	Boolean	Owner is verified	Repository feature
Issue		New issue added	Event
languages	List	Programming languages used	Repository feature
Pull Request		Pull request created	Event
Push		Commit pushed	Event
Release		New release added	Event
Stargazers		Stargazer added	Event
Subscribers		Subscription added	Event
Watchers		Repository watched	Event

Table 2
Features collected from GH Archive.

Feature	Description
Create	A Git branch or tag was created
Delete	A Git branch or tag was deleted
Issue Comment	A comment on an issue was added
Commit Comment	A comment on a commit was added
Pull Request Review Comment	A comment on a pull request was added
Member	Membership on the repository changed
Public	A repository changed its viewing settings
Push	A commit was pushed to the repository
Fork	The repository was forked
Release	A new version of the repository was released
Issue	A new issue was added
Watchers	User marked the repository was watched
Pull Request	A Pull request was created

messages serve as a means for developers to articulate the essence of code modifications. While some repositories enforce text patterns or guidelines for these messages to maintain consistency and clarity, the majority do not, leading to variability in message formats across different projects.

3.4. Code patch preprocessing

A patch comprises of two main components: a list of added lines and a list of deleted lines, for each modified file. Encoding this information can be approached in several ways. We experimented with the following methods:

(1) *Concat Encoding* — Concatenating all additions and deletions, respectively, into a single sentence.
(2) *Concat with Comment Encoding* — Similar to Concat Encoding, combining additions and deletions into a single sequence. However, here, a line comment matching the code language used was added before each deletion. This maintains the original syntactic meaning of a deleted line, as any compiler/interpreter would discard it. This technique is exemplified in Fig. 3, where conventional Git diff displays a removed line, while comment encoding exhibits an added line representing a comment for the removed line. Consequently, encoding deleted or added that are already a comment preserves their intended meaning.
(3) *Special Token Encoding* — Inserting special tokens, $\langle ADD \rangle$ and $\langle DEL \rangle$, before each line of added and deleted code, respectively.
(4) *Sequence Match Encoding* — Instead of utilizing added and deleted lines, this method involved collecting the file's state before and after the modifications. We employed the *SequenceMatcher* library from *difflib*, which employs the Gestalt Pattern Matching algorithm (Ratcliff and Metzener, 1988). This process yielded a list of changes, additions, and deletions (in addition to a list of unchanged strings, which we disregarded). Subsequently, we processed the data in a similar manner to the *Special Token Encoding*, employing separate encoding for each list.

It is important to note that a single patch can encompass changes across multiple files simultaneously. Therefore, to classify a commit as a security patch, for each method we concatenated the encoding of all files together.

3.5. Beahvioral data pre-processing

A temporal *window* is a configurable parameter, which denotes the number of events before and after the security patch commit that are concatenated together to create a window (or a vector) of features. This window of features is one of many that will be fed into the model to train it.

Although the time gap between events provides limited insight for the task, other temporal features such as hour, day of the week, and month were found to be informative. These features, along with event types, were encoded using one-hot encoding and input into the time series model, which typically processes numerical data rather than categorical events. Next, we gather for each repository all its security patch commits. As mentioned above, to make the dataset balanced, we also collected a random sample of non-security patch commits. These will be used as labels for our model.

For each repository, we created a list of events with their date and time. Commit events also have additional normalized integer features: the number of lines added to a file, the number of lines deleted from a file, and the number of files modified. Furthermore, repositories that contained less than 100 events overall were discarded since they were mostly insufficient for extracting the windows that we required.

4. Methods

In this section, we describe the process of creating and training the models with the data described above. As mentioned earlier, we trained three separate sub-models and combined them into a single composed model. Hence, we collect the necessary data for each individual model.

```
/* Git Diff */
- memcpy(dst,src,len);
+ if(len<sizeof(dst))
+     memcpy(dst,src,len);

/* Git Diff after comment encoding */
+  // memcpy(dst,src,len);
+  if(len<sizeof(dst))
+    memcpy(dst,src,len);
```

Fig. 3. Comment encoding example.

4.1. Methodology definition

In this section, we present our methodology for addressing the aforementioned problem of classifying commits as security patches. An overview of the process is illustrated in Fig. 1.

A commit, denoted as C_i, can be uniquely identified by its commit hash, repository name, and the repository owner's name. It is composed of the following data components:

1. $\mathbf{b}_i \in \mathbb{R}^{d_b}$ — Behavioral data: This encompasses all events that occurred before and after the commit. Where the dimensionality $d_b \times 401$, where the first dimension denotes the window size — a hyper-parameter, as detailed in Section 5.1, and the second is the number of behavioral features in each window (401). Further details regarding the behavioral data will be provided below.
2. $\mathbf{t}_i \in \mathbb{R}^{d_t}$ — Textual commit message: A textual field that provides a summary or description of the commit. The dimensionality $d_t = 510$ denotes the size of the embedding of a token (word).
3. $\mathbf{c}_i \in \mathbb{R}^{d_c}$ — Code patch: For each file in the repository, a list of code lines that were added or deleted as part of the commit. The dimensionality $d_c = 510$ is the size of the embedding of a code token.
4. s_i — A scalar binary label denoting whether the commit is a security patch.

Formally, we define the single modality datasets: behavioral, commit message, and code patches as $\{\mathbf{b}_i, s_i\}_{i=1}^N \in \mathcal{D}_b$, $\{\mathbf{t}_i, s_i\}_{i=1}^N \in \mathcal{D}_t$, and $\{\mathbf{c}_i, s_i\}_{i=1}^N \in \mathcal{D}_c$, respectively, where N represents the total number of collected commits. The multi-modal dataset is defined as $\{\mathbf{b}_i, \mathbf{t}_i, \mathbf{c}_i, s_i\}_{i=1}^N \in \mathcal{D}_m$.

4.1.1. Single-modality models

As a first stage of our approach, we build separate single modality prediction models for the behavioral, commit messages, and code patches. Each of the single modality models is a concatenation of two neural networks f_θ and g_ϕ parameterized by θ and ϕ, respectively. The first network, g is designed to take the single modality data and transform it into a latent representation which is then used by the second network f_θ to produce the final prediction. Hence, we define the following single-modality prediction models as follows:

$$\begin{aligned} &f^b_{\theta_b}(g^b_{\phi_b}(\mathbf{b}_i)) = \hat{s}^b_i, \\ &f^t_{\theta_t}(g^t_{\phi_t}(\mathbf{t}_i)) = \hat{s}^t_i, \\ &f^c_{\theta_c}(g^c_{\phi_c}(\mathbf{c}_i)) = \hat{s}^c_i, \end{aligned} \tag{1}$$

where $g^b_{\phi_b}$, $g^t_{\phi_t}$, and $g^c_{\phi_c}$ take the behavioral, commit message, and code patches separately and transform it into single modality representation views. These single-view representations are then processed by $f^b_{\theta_b}$, $f^t_{\theta_t}$, and $f^c_{\theta_c}$ to generate the independent single modality predictions $\hat{s}^b_i$, $\hat{s}^t_i$, and $\hat{s}^c_i$ for the behavioral, text message, and code, respectively. Note that this breakdown of each single-modality prediction model into a concatenation of two networks g_ϕ and f_θ is unnecessary at this stage, but will be utilized later in the final multi-modality solution.

The single-modality prediction models are then trained separately to extract the relevant information in each modality for the sake of identifying security patches. We employed the cross-entropy loss defined as:

$$\mathcal{L}_{ce}(\hat{y}, y) = -y \cdot \log(\hat{y}) - (1-y) \cdot \log(1-\hat{y}) \tag{2}$$

where $\hat{y}$ and y are the prediction and the binary ground-truth label, respectively. The single-modality models are initialized with random parameters and trained separately according to the following objectives:

$$\begin{aligned} &\underset{\theta_b,\phi_b}{\operatorname{argmin}} \frac{1}{N} \sum_{(\mathbf{b}_i,s_i)\in\mathcal{D}_b} \mathcal{L}_{ce}\left(f^b_{\theta_b}(g^b_{\phi_b}(\mathbf{b}_i)), s_i\right), \\ &\underset{\theta_t,\phi_t}{\operatorname{argmin}} \frac{1}{N} \sum_{(\mathbf{t}_i,s_i)\in\mathcal{D}_t} \mathcal{L}_{ce}\left(f^t_{\theta_t}(g^t_{\phi_t}(\mathbf{t}_i)), s_i\right), \\ &\underset{\theta_c,\phi_c}{\operatorname{argmin}} \frac{1}{N} \sum_{(\mathbf{c}_i,s_i)\in\mathcal{D}_c} \mathcal{L}_{ce}\left(f^c_{\theta_c}(g^c_{\phi_c}(\mathbf{c}_i)), s_i\right). \end{aligned} \tag{3}$$

This optimization is achieved via stochastic gradient descent using the Adam optimizer (Kingma and Ba, 2014).

4.1.2. Multi-modality prediction

In order to utilize multi-modality data, we take the single-modality representation learning models from the previous step, namely $g^b_{\phi_b}$, $g^t_{\phi_t}$, and $g^c_{\phi_c}$ as input to a new multi-modality prediction network $f^m_{\theta_m}$. Formally, given a multi-modal datum $\{\mathbf{b}_i, \mathbf{t}_i, \mathbf{c}_i, s_i\}$ we construct our multi-modality model as follows:

$$h_\theta = f^m_{\theta_m}\left(g^b_{\phi_b}(\mathbf{b}_i), g^t_{\phi_t}(\mathbf{t}_i), g^c_{\phi_c}(\mathbf{c}_i)\right) = \hat{s}^m_i, \tag{4}$$

where $f^m_{\theta_m}$ is the multi-modality prediction model that takes representations from the single modality models $g^b_{\phi_{bm}}$, $g^t_{\phi_{tm}}$, $g^c_{\phi_{cm}}$ to generate the multi-modality prediction $\hat{s}^m_i$.

Given the multi-modality dataset $\mathcal{D}_m$, the multi-modality model is optimized in an end-to-end manner according to:

$$\underset{\theta,\phi_b,\phi_t,\phi_c}{\operatorname{argmin}} \frac{1}{N} \sum_{\mathcal{D}_m} \mathcal{L}_{ce}\left(h_\theta\left(\mathbf{b}_i, \mathbf{t}_i, \mathbf{c}_i\right), s_i\right). \tag{5}$$

We found that this optimization is improved if instead of starting from random single modality view networks, we employ the optimized single view networks from above, namely $g^b_{\phi_b}$, $g^t_{\phi_t}$, and $g^c_{\phi_c}$. However, it is important to note that by training in an end-to-end manner, each single modality view network is further updated to support the missing information in the other modalities. In the evaluation section, we shall demonstrate the importance of this end-to-end training step and its advantage over a simple composition of pre-trained models without end-to-end training.

The methodology outlined above forms the basis of our approach to classify commits as security patches.

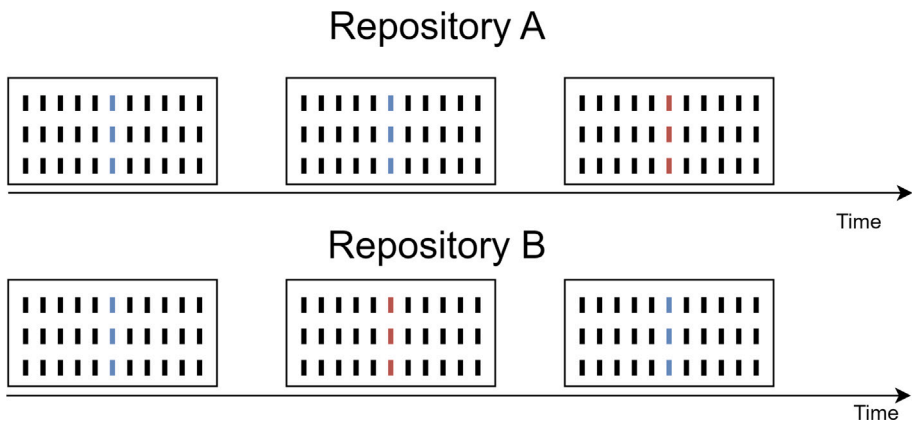

Fig. 4. Demonstration of the sliding window method.

4.2. Models

In what follows, we describe our implementation of the single modality sub-models that make up PatchView.

4.2.1. The behavioral time series modality

Different types of architectures were examined for modeling the behavioral modality i.e., for g^b_ϕ: (1) LSTM is an architecture that had proven its efficiency for such tasks. LSTM (Hochreiter and Schmidhuber, 1997) is based on Recurrent Neural Network (RNN) architecture, where performance decreases as a larger number of time steps are fed into the network. However, LSTM can forget some less important data, while preserving the more important parts of it. (2) GRU (Cho et al., 2014) have an architecture that is similar to LSTMs but with fewer parameters, which may be an advantage for some learning tasks. Finally, (3) Conv1D is a Convolutional Neural Network (CNN) that convolutes 1-dimensional data, such as time series. This model can be stacked to allow multi-level pattern recognition.

These architectures receive input data in the form of a series of vectors, each may be associated with many features. As previously stated, the dimensionality of the data input to the model is $d_b \times 401$, where d_b represents the window size and 401 denotes the number of features. The model's output dimensionality is a single prediction, $\hat{s}^t_i$.

The following process, as can be seen in Fig. 4 is employed to fit a time series as input to the mentioned architectures. For each repository, in our dataset, we gathered all events that surround each commit, as a single input to the model. For example, an 11-event window (composed of a certain event, 5 events before and 5 events after) is classified as a one that contains a security patch (marked with a red column) or blue (as a window with non-security patches). As can be seen in the figure, some events may contain more than one feature (one column is composed of a few short lines). Window size is the number of events that are contained in the window, each window size is constant, and windows are fed into the models one by one. It should be noted that the number of events preceding a commit is not necessarily equal to the number of events following it. It is important to note that, since this sub-model does not base its decisions on the code or commit message, it remains independent of these factors.

4.2.2. The commit message modality

For processing the commit message by g^t_ϕ, we employed RoBERTa (Liu et al., 2019), a language model based on the BERT (Devlin et al., 2018) architecture. RoBERTa is trained on a substantial corpus of publicly available text sourced from the internet, and its training focuses on effectively comprehending and generating natural language text. These carefully chosen NLP models are expected to offer effective and versatile solutions for their respective tasks in our study. We used multilingual RoBERTa to make sure that even commits messages are in different languages than english, we would still be able to use them.

4.2.3. The code modality

For the code modality, we implemented g^c_ϕ using CodeBERT (Feng et al., 2020), a variant of the BERT transformer architecture, which has been trained on an extensive dataset comprising publicly available code from various open-source repositories. Despite being trained exclusively on six programming languages (Python, Java, JavaScript, PHP, Ruby, and Go), CodeBERT is adapted for classifying code from other programming languages (Chen et al., 2023).

4.2.4. Composed model

The composed model h_θ leverages diverse insights provided by the individual sub-models and makes a final classification decision, denoted as $\hat{s}^m_i$ in Eq. (4). Next, we discuss several alternatives for utilizing multi-modal data.

1. *Averaging Ensemble* — This configuration involves aggregating the probabilities generated by all sub-models, summing them, and subsequently selecting the label with the highest summed probability. In other words, each sub-model's prediction contributes to the final decision through a voting mechanism.
2. *Frozen Sub-models* — In this alternative, we do not train the multi-modality model in an end-to-end manner. Instead, the sub-models remain frozen in the training of the multi-modality model. Formally, the multi-modal approach is given by $h_\theta = f^m_{\theta_m}\left(g^b_{\phi_b}(\mathbf{b}_i), g^t_{\phi_t}(\mathbf{t}_i), g^c_{\phi_c}(\mathbf{c}_i)\right)$, where only θ_m is optimized while $g^b_{\phi_b}$, $g^t_{\phi_t}$, and $g^c_{\phi_c}$ are kept frozen, namely ϕ_b, ϕ_t and ϕ_c remain untouched.
3. *End-to-end training* — Our suggested solution, namely PatchView, which is explained thoroughly in Section 4.1.2.

It is also important to highlight that, since each sub-model is capable of processing patches regardless of the programming language or natural language, the overall model is consequently able to classify patches irrespective of the language in which they are written.

5. Experimental design

The experiments detailed in this study utilize the dataset introduced in Section 3.1. Unlike preceding research efforts that did not make their datasets public or as comprehensive as ours, this paper's dataset is openly accessible alongside our open-source code in our Git repository. Given the scarcity of publicly available datasets for CVE detection, this accessibility represents an additional contribution of our work, making it easier to compare against alternative approaches and future improvements.

Remember that our dataset is balanced as mentioned in Section 3.1. In the initial phase, we randomly partitioned the commits into training and test sets, constituting 90% and 10% of the total commits, respectively. We then applied 10-fold cross-validation to the training

set to fine-tune the hyper-parameters. Using this dataset, we assessed the PatchView model's effectiveness and performance in CVE detection. We also benchmarked our method against current leading CVE detection approaches, demonstrating a new state-of-the-art results for the PatchView model. Furthermore, we present an extensive ablation study that highlights the individual contributions of the model's components and the relative importance of each data source.

5.1. Hyper-parameters

In this section, we describe PatchView's hyper-parameters and their tuning procedure. We used a Bayesian hyper-parameter search with Hyperband Li et al. (2018) Algorithm for our hyper-parameter tuning process. The hyper-parameters are as follows:

- **Dropout** — This is a value between 0.2 and 0.8 which was used to signal all neurons across all sub-models to ignore a percentage of the data they receive.
- **Learning Rate** — We used different learning rates for the models, possible values were $\eta \in [1e-4, 5e-5, 3e-5, 1e-5]$
- **Weight Decay** — We used a range of weight decay values to penalize the model's complexity, the experimented values were $[0.01, 0.001, 0.0001]$.
- **Batch size** — We used different scale of batch sizes for the different modalities, as NLP models are capable of handling smaller batch sizes. Code and text models used batch sizes in the range of $[8, 16, 32]$ whereas the behavioral model was experimented with batch sizes of $[128, 256, 512]$
- **Behavioral model window size** — As mentioned in Section 4.2.1, different number of gathered events before and after the classified event can be chosen. In our case, the number of events before and after the classified event were in the range of 5 to 40.
- **Behavioral model layer sizes** — The behavioral model contained 3 temporal layers (Conv1D/ LSTM/ GRU). Each one had a different number of neurons in it ranging between 100 and 1024 neurons on the first layer and 16 to 128 in the second layer and the third layer.

5.2. Experiment implementation

The proposed models were implemented using the PyTorch library (Paszke et al., 2019) in Python. To facilitate the training process, we used a Docker engine that encompassed an NVIDIA RTX A5000 GPU, providing significant computational power for accelerated training. The GPU possessed 50 GB of memory, enabling efficient processing of large-scale deep learning models.

The training time varied across the models. For the code and message model, each training epoch required approximately 4 min to complete. The behavioral model had a rapid training process, with each epoch taking less than a second to finish. Finally, training the composed model necessitated approximately 8 min to complete a single epoch.

6. Results

First, we outline the metrics used to evaluate our models, followed by an examination of our multi-modal architecture results. Next, we analyze the importance of different features utilized by our sub-models and present an ablation study. Finally, we present a case study demonstrating the application of our models

6.1. Evaluation metrics

The performance of the trained models is evaluated using metrics tailored for binary classification tasks. These include:

- **Accuracy**: Measures the proportion of true results (both true positives and true negatives) among the total number of cases examined. It provides a quick overview of model performance but can be misleading in imbalanced datasets.
- **Precision**: Indicates the proportion of positive identifications that were actually correct. Precision is crucial when the cost of false positives is high.
- **Recall** (Sensitivity): Measures the proportion of actual positives correctly identified. It is important when the cost of false negatives is significant.
- **F1-score**: A harmonic mean of precision and recall, balancing the trade-off between them. It is particularly useful when one needs to compare two models with similar accuracy but different precision and recall.
- **ROC Curve**: Plots the true positive rate against the false positive rate at various threshold settings, providing insight into the trade-off between sensitivity and specificity. The area under the ROC curve (AUC) is a comprehensive measure of the model's ability to distinguish between classes.

These metrics collectively offer a holistic view of model performance, capturing both the effectiveness in identifying positive cases and the accuracy of predictions across classes.

Table 3
Composed model accuracy under different configurations.

Name	End-to-end training	Accuracy
Averaging	–	88.7
Frozen sub-models	✗	87.3
PatchView	✓	92.5

Table 4
Average training time for each model, measured in seconds.

Model	Average training time (s)
Behavioral Model	491
Code Model	3,275
Message Model	2,536
Frozen Sub-Models	1,954
PatchView	2,017

6.2. The composed multi-modal model

In this section, we present the results of the multi-modal model, which is composed of the three sub-models mentioned in Section 4.1.1. Initially, we trained each sub-model separately, selecting the most suitable hyper-parameters for each, as described in Section 6.4. Then, we experimented with ways to combine the three sub-models, as tabulated in Table 3 and explained in Section 4.2.4. The averaging approach aggregates probabilities from all sub-models, summing them to select the label with the highest probability, achieving an accuracy of 88.7%. The frozen sub-models method, which prevents further weight adjustments by freezing all sub-model weight, after training each alone, resulted in an accuracy of 87.3%. However, when sub-models were not frozen and trained end-to-end as part of the ensemble, PatchView achieved an accuracy of 92.5%. This highlights the importance of end-to-end training, where combining different modalities allows for better classification than single-modality approaches. Furthermore, when training the multi-model without initial training of each sub-model, the multi-model was not able to converge at all. Average training times of each of the sub-models and multi-model configurations can be shown in Table 4.

Table 5
Confusion matrix of the classification of patches in the test set.

	Not classified as a security patch	Classified as a security patch
Not Marked as a Security Patch	400 (TN)	41 (FP)
Marked as a Security Patch	30 (FN)	488 (TP)

6.2.1. Evaluating patchview accuracy

The confusion matrix in Table 5 demonstrates the performance of PatchView. The model achieved an accuracy of 92.5%, with precision, recall, and F1-scoreof 92.2%, 94.2%, and 93.2%, respectively. The confusion matrix shows the model's ability to correctly classify most instances, while indicating that false positives are the model's primary weakness.

To gain deeper insights into our results, we manually examined the test set output, focusing specifically on the 41 false positives (FPs). This analysis revealed that 19 out of the 41 cases (46%) indeed contained security patches. Reclassifying these 19 cases as true positives (TPs) in the confusion matrix presented in Table 5 leads to a revised accuracy of 94.5%. This adjustment also improves the precision, recall, and F1-score to 95.8%, 94.4%, and 95.1%, respectively. Few examples of patches that were initially labeled as FP, but manually identified as TP are detailed in Section 6.5.

6.2.2. Comparison with previous works

Table 6 provides results of a comparative analysis of our findings with those of previous studies that have aimed to detect security patches. The results from these previous studies were reported in their respective papers. Our study uniquely incorporates developer behavioral events, commit messages, and code patches as integrated features, distinguishing it from existing literature in the field. Additionally, our study benefits from a significantly larger dataset, encompassing three times more commits for training and testing purposes. This expanded dataset also encompasses a more diverse range of security patches, originating from various programming languages and repositories. When evaluating the outcomes, it is evident that PatchView surpasses all other studies in terms of accuracy and F1-score.

To enable direct comparison of results, we trained a version of PatchView on the same dataset used by Nguyen et al. (2022), the current state-of-the-art. Analysis of the results from this experiment indicates a notable 6% improvement in the F1-scoreover (Nguyen et al., 2022).

It is worth noting that this improvement can be attributed to several key factors. Firstly, our model is trained on a substantially larger dataset of security patches, in contrast to Vulcurator, which relies on a manually-curated dataset of security patches and the TensorFlow repository exclusively. Additionally, for code patches our model incorporates the novel *Concat with Comment* technique, which enhances its ability to comprehend the patches more effectively, as discussed in Section 3.4. Lastly, we have demonstrated that the incorporation of a multi-modal model, particularly the use of a behavioral sub-model, significantly enhances the accuracy of the predictions, as detailed in Section 6.4.

6.2.3. Analyzing model performance using ROC curve

Fig. 5 illustrates a Receiver Operating Characteristic (ROC) curve. The Area Under the Curve (AUC) is 0.97, indicating strong discriminatory capacity in the model's binary classification performance. In practical terms, when a security researcher examines our model's output, they can rely on the model to correctly identify security patches with minimal false positives, thereby enhancing the efficiency and effectiveness of their work. This high level of accuracy can significantly reduce the time and resources spent on manual verification.

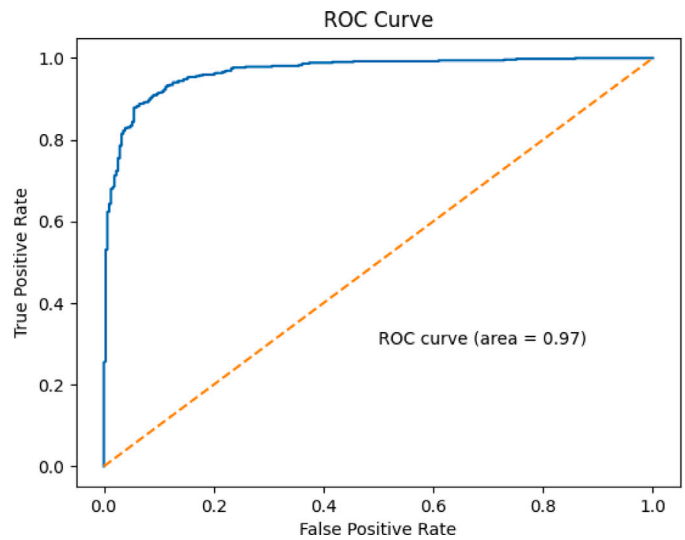

Fig. 5. ROC curve for the best-performing model, achieving an Area Under the Curve (AUC) of 0.97. The optimum, which is the point with the smallest euclidean distance to the point (1,0) is 0.1 FPR and 0.9 TPR.

6.3. Model feature importance analysis

In this section, we analyze the significance of the features employed by our models in classifying security patches. We begin with an overview of the model's accuracy across different repositories and programming languages. Next, we use Shapley (Lundberg and Lee, 2017) values to derive insights from the commit message model, identifying key terms for classification. Finally, we examine the behavioral model's feature importance to elucidate the primary factors influencing predictions. These findings provide critical insights into the models' strengths and limitations in detecting security patches, informing future enhancements in software security.

We start by delving into the performance of the models across different repositories and their ability to identify security patches accurately. Table 7 provides an overview of the classification accuracy for PatchView across various repositories. The table reveals that the model exhibit proficiency in identifying security patches for repositories with a substantial number of instances. The detection of 100% of the security patches in the Linux kernel, which were part of the test-set, is notable since other studies only attained a moderate score for this task (Tian et al., 2012; Zhou et al., 2021). This achievement is particularly significant given the prominence of the Linux kernel. For one repository, "radare/radare2", the accuracy is significantly lower than for others. As discussed in Section 6.5, we discovered that some commits in this and other repositories were incorrectly labeled as non-security patches when they were actually security patches. After relabeling these commits in this repository, the accuracy for "radare/radare2" reached 100%.

Next we examine the classification accuracy for various programming languages. Since the Commit Message Model is the one sensitive to the programming language Table 8 shows the accuracy of this model alone. Notably, languages such as C, PHP, and Python achieve high accuracy rates of 0.91, indicating the model's effectiveness in identifying security patches in these languages. It is worth mentioning that the model achieves a perfect accuracy score of 1 for languages such as Perl and C#, although they appear in a very small share of all commits. We speculate that the reason for the difference in accuracy between C and C++ is because C++ is a more complex language compared to C. This is mainly because C++ has additional features such as object-oriented programming, RAII, and exception handling that make it more intricate than C and harder for a model to understand.

Table 6
Comparison of results with previous studies.

Study	Modalities	Used models	# CVE Commits	Accuracy	F1-score
PatchRNN (Wang et al., 2021b)	Commit messages, code	LSTM	4,076	83.57%	74.7%
SPI (Zhou et al., 2021)	Commit messages, code	LSTM	1,045	–	87.93%
Sun et al. (2020)	Commit messages	HAN	3,704	92.81%	–
Zuo et al. (2023)	Commit messages	Transformer	6,226	91.7%	89.1%
Vulcurator (Nguyen et al., 2022)	Commit messages, issues, code	RoBERTa, CodeBERT	8,952	–	89%
PatchView on Vulcurator's dataset (Nguyen et al., 2022)	Behavioral data, commit messages, code patches	RoBERTa, CodeBERT, Conv1D	8,952	–	95.71%
PatchView	Behavioral data, commit messages, code patches	RoBERTa, CodeBERT, Conv1D	24,417	**94.52%**	**95.12%**

Table 7
Model classification accuracy per repository.

Repository name	Accuracy	Share of all repositories
torvalds/linux	1.00	0.12
phpmyadmin/phpmyadmin	0.91	0.03
wordpress/wordpress	0.95	0.02
imagemagick/imagemagick	1.00	0.02
gpac/gpac	0.93	0.02
php/php-src	0.93	0.02
radare/radare2	0.83	0.01
livehelperchat/livehelperchat	0.90	0.01
limesurvey/limesurvey	1.00	0.01
ffmpeg/ffmpeg	1.00	0.01
⋮	⋮	⋮

Table 8
Model classification accuracy per language.

Language	Accuracy	Share of all commits
C	0.91	0.391
PHP	0.91	0.244
JavaScript	0.89	0.080
Java	0.84	0.062
Python	0.93	0.059
C++	0.82	0.048
Ruby	0.93	0.022
TypeScript	0.75	0.016
HTML	0.9	0.013
Perl	1	0.009
Go	0.85	0.009
C#	1	0.005
⋮	⋮	⋮

In general, certain programming languages are more prone to exhibiting complex patterns than others (Pan et al., 2009), as described above regarding C and C++. Moreover, certain languages may be entirely devoid of specific types of bugs, thus lowering the probability of encountering such issues. For instance, memory corruption bugs are less prevalent in Python (Ray et al., 2014; Abuwala and Lim). These underlying principles likely contribute to the differences in bug distribution across various programming languages.

The model's capability to accurately classify security patches for programming languages beyond those it was specifically trained on, such as C, C++, and TypeScript is particularly noteworthy. This finding suggests that the model can leverage shared similarity and key programming language keywords to effectively classify security patches, even in unfamiliar languages. Overall, the table provides valuable insights into the model's accuracy in different programming languages,

Table 9
Terms that the commit message model indicated important for classification as security patch. The "Occurrences" column indicates the frequency of each term in commit messages, based on an analysis of approximately 27,130 commits.

Word	Effect on decision (Shapley values)	Occurrences
Sanitizer	0.88	16
Escaping	0.87	65
Malformed	0.82	41
Sensitive	0.78	30
Security	0.75	533
Escapes	0.74	6
HTTPS	0.73	36
Reject	0.68	71
disallow	0.66	52
UAF	0.61	26
Escape	0.61	150
Sanitized	0.61	16
Restricted	0.6	36
Paths	0.57	142
Detection	0.57	109
validate	0.53	166
Alternative	0.52	43
Safe	0.49	101
Malicious	0.48	89
Filtering	0.48	42

highlighting its capability to detect security patches across a wide range of language frameworks. Note that the single-modality CodeBERT model which represents $g^c_{\phi_c}$ is able to produce representations to only a few languages (Python, Java, JavaScript, PHP, Ruby, Go) before training on our dataset.

To enhance our understanding of the commit message model's operation, we calculated Shapley values as an additional explainability metric. Table 9 lists words identified by the model as significant for classifying security patches, each accompanied by its Shapley value, reflecting its influence on the decision-making process. The table also includes the frequency of each word and their relative proportions among all commit message words. As shown, most of these words are infrequently used in commit messages, highlighting the complexity of the problem that cannot be resolved through simple rule-based solutions.

The table highlights several significant words, such as "Sanitizer", "Escaping", "Malformed", and "Sensitive" which possess relatively high Shapley values, ranging from 0.78 to 0.88. These words suggest a strong association with security-related concerns and play a crucial role in determining the classification of commits as security patches. For example, a sanitizer is an automatic tool that is able to detect memory corruptions, "escaping" is a word that usually describes the ability to inject unintended characters over filtering rules, and "malformed"

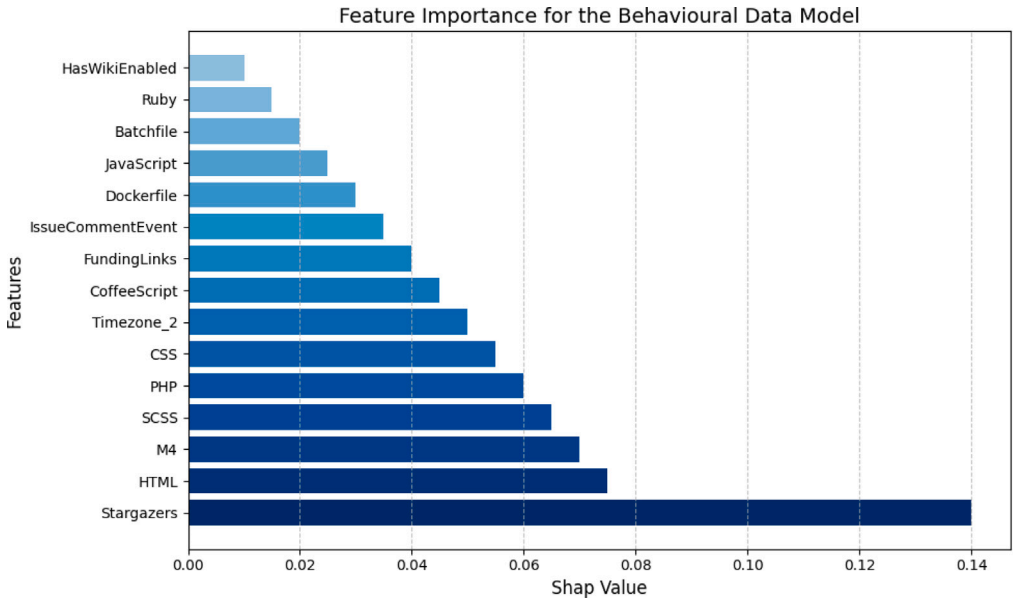

Fig. 6. Feature importance for the behavioral model, where the most important feature is stargazing a repository (i.e., bookmarking a repository on GitHub to indicate interest or appreciation).

usually describes data that does not stand in the regular conditions of a program and might cause security issues.

Other notable security related words include "HTTPS", "Reject" and "UAF" (use after free - a type of bug class) have high Shapley values between 0.61 and 0.73. Overall, this table provides insights into the key terms that contribute significantly to the commit message model's ability to identify security-related patches.

To gain deeper insights into the behavioral model's internal operations, we computed Shapley values (Lundberg and Lee, 2017) to examine the impact of various features on the model's decisions. Currently, calculating Shapley values for time series models can only determine the influence of individual features on the decision, but not the impact of consecutive features and their order. In Fig. 6, we present a comparative analysis of the 15 most influential features and their corresponding Shapley values. In addition, we observe that the feature `timezone_-7`, representing operations performed in timezone -7, was identified as the least informative feature for the model.

Among the significant features, "stargazers" signifies the act of "liking" or following a repository. "issue" represents the number of issues associated with the repository. "issue comment event" denotes the occurrence of a comment on an issue. Finally, "timezone_2" indicates whether the commit was made in UTC+2 time zone. We suspect this is because UTC+2 hosts two nations, Israel, and Russia, with a significant IT sector that is heavily biased towards security. Thus, the portion of security related patches from UTC+2 is expected to be significantly higher than the world average. Additionally, it is evident that the behavioral model's decision is influenced by the programming language used in the code. We suggest that changes to some files such as HTML and PHP are indication that there are or there are no vulnerabilities in the current commit.

6.4. Ablation study

A comparison of individual sub-models with the composed model is presented in Table 10. Results indicate that the composed model surpasses all sub-models, exhibiting superior accuracy, precision, recall, and F1-score. We employed a composite models that excludes each of the modalities at a time. Although these model demonstrates higher accuracy compared to other sub-models, it remains less accurate than the full composite model, which benefits from the additional insights provided by third modality. This underscores the effectiveness of integrating diverse models and modalities in enhancing overall classification performance. Moreover, after refining the labels, detailed in Section 6.5, these metrics show further improvement.

Table 10
Comparison of results of different modalities.

Modalities	Accuracy score	F1-score
Code patches	82.34%	83.33%
Behavioral data	83.93%	84.14%
Commit messages	88.71%	89.05%
Behavioral data and Code patches	85.88%	86.65%
Commit messages and code patches	90.25%	90.61%
Commit messages and Behavioral data	91.19%	92.21%
Behavioral data, commit messages and code patches	92.51%	93.22%

As detailed in Section 4.2, we explored three architectures for modeling the behavioral aspect: LSTM, GRU, and Conv1D. These models underwent evaluation across a spectrum of window sizes, with the Conv1D architecture consistently surpassing its counterparts in every configuration. Fig. 7 illustrates the accuracy comparison among the three models, using an optimal symmetric window size of 20. The Conv1D model demonstrated a significant advantage, outperforming the next closest model by three percentage points, thereby affirming its selection for further experiments. Conv1D architectures afford a comprehensive perspective of input data via fully connected layers. Conversely, LSTM, and GRU networks lack simultaneous insight into the entirety of the window.

Next, we evaluate the impact of window size, a critical hyper-parameter for the behavioral model. This hyper-parameter determines the number of preceding and following commits considered relative to the current commit under evaluation. To this end, the performance of the Conv1D model across various window sizes is depicted in Fig. 8. Notably, the model achieved peak accuracy with a window size of 15 for preceding events and 5 for following events. Smaller window sizes were associated with higher accuracy, suggesting that larger windows might introduce irrelevant information, thereby diminishing the model's efficiency in identifying relevant patterns and features. A reduced window size appears to sharpen the model's focus on pertinent data, enhancing its classification performance.

As discussed in Section 3.4, various code encoding techniques were employed to facilitate the CodeBERT model in classifying additions and

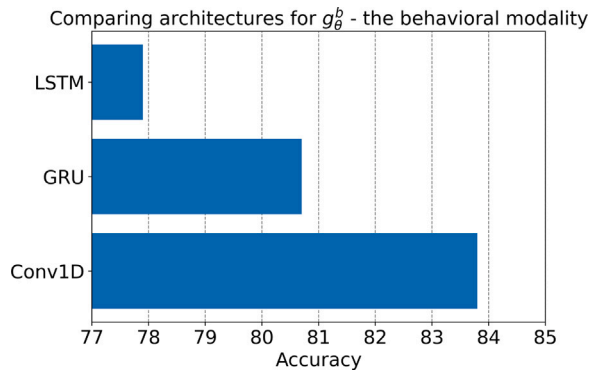

Fig. 7. Comparison of model accuracy for different architectures.

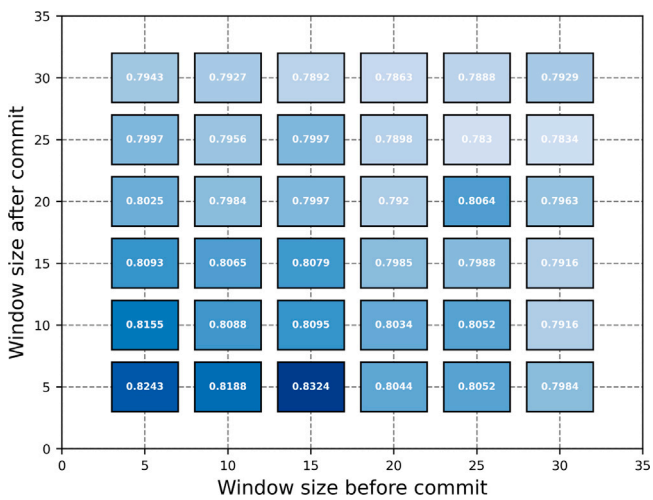

Fig. 8. Effect of the window size before and after the security patch on model accuracy using the Conv1D model.

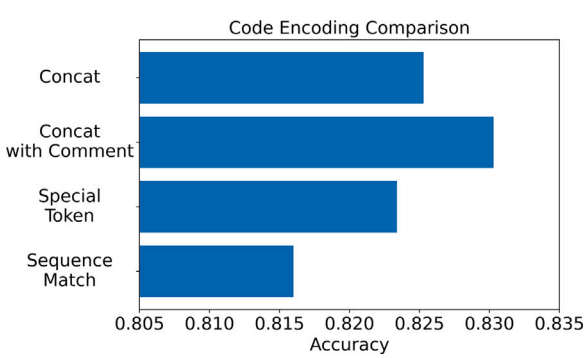

Fig. 9. Maximal accuracy per code patch encoding type.

deletions within commits as security patch commits. Notably, aside from the foundational *Concat* method, the remaining three methodologies are first suggested here and possess potential usage for other code analysis applications. Fig. 9 shows that the encoding methodology that outperformed the others was the *Concat with Comment Encoding* technique.

This outcome can be attributed to the likelihood that encoding deleted lines with comments contains tokens that NLP models such as CodeBERT are more accustomed to, given that comments are prevalent and convey similar semantics to the action of deleting a line.

6.5. Case study

As discussed above, some vulnerabilities are fixed without being reported as security patches. In this case, our model will classify them as security patches, although they are not be labeled as such. As a result, during our evaluation they will be marked as false-positive. We manually scrutinized the false-positive samples for the presence of security patches. Among the 41 false-positive commits analyzed, we detected 19 instances containing security patches, constituting 46% of the total. While it is not feasible to detail all these 19 security patches here, we provide an in-depth examination of six case studies. We summarize the detected security patches in Table 11. To understand better how the model could improve even more, we analyzed the cases the model fails as well, 2 of those cases could be viewed at Appendix A

`LibTIFF` is a library specifically designed for handling the TIFF image format. A notable commit[8] addressed two critical heap-based buffer overflow vulnerabilities. Although this patch significantly enhances security, it has not been assigned a CVE identifier. Instead, it is tracked under a Bugzilla issue,[9] which our model does not consider for labeling.

Remarkably, this security patch was successfully identified as such by all of our analytical models: the code model, the commit message model, the behavioral data model, and the composed model. Despite its recognition across these models, the absence of a CVE assignment for this commit highlights potential gaps in the vulnerability documentation processes.

`Wireshark` is a network protocol analyzer that allows viewing network communication at several protocol layers. A Commit[10] fixed a denial of service attack by null pointer dereference — which can be used to neutralize the software, enabling subsequent attacks to proceed unnoticed. This security patch was successfully identified as such by all of our analytical model as well.

`Radare2`, a platform designed for reverse engineering, facilitates the comprehensive analysis of binary files. The commit[11] addresses a security vulnerability stemming from inadequate validation of buffer sizes for Unicode strings, leaving the system susceptible to buffer overflow attacks. Despite being detected by all analytical models, including the code, commit message, behavioral data, and composed models, the commit did not receive a CVE identifier.

`Radare2 -- Security patch 2` Commit[12] within the Radare2 repository, a security patch was applied to address an out-of-bounds read vulnerability occurring during the reading of a buffer in one of the fields of a DEX file. Despite being detected by both the code modality model and the commit message modality model, this particular commit was not associated with a CVE identifier.

`SLPJS` is a library for validating and building token transactions. This commit[13] contains added validation for tokens sent to the website. This commit was detected by the behavioral data model, but was not assigned a CVE.

`Path` repository from the Aurelia group contains utilities for path manipulation. Commit[14] introduced a security patch addressing Prototype pollution, a JavaScript vulnerability enabling attackers to append

[8] https://GitHub.com/vadz/libtiff/commit/5ed9fea523316c2f5cec4d393e4d5d671c2dbc33
[9] http://bugzilla.maptools.org/show_bug.cgi?id=2633
[10] https://GitHub.com/wireshark/wireshark/commit/ba179a7ef7e660d788cbaade65982ffc7249b91f
[11] https://github.com/radareorg/radare2/commit/599cbcd3c796207e8c5af23f0947186a2055f8c8
[12] https://github.com/radareorg/radare2/commit/bb7672c6a5fc2b94f39b5dbcf9f416336ad88bc0
[13] https://github.com/simpleledger/slpjs/commit/de5cf37815ca3b7ba872572b3387388d54dcdcc9
[14] https://github.com/aurelia/path/commit/d3bb6faa8180cf245a15f7754f52b5acd9178ef1

Table 11
Summary of false positives examples that actually contained security patches.

Repository name	Code model	Commit message model	Behavioral data model
LibTIFF	✓	✓	✓
Wireshark	✓	✓	✓
Radare2 (1)	✓	✓	✓
Radare2 (2)	✓	✓	✗
slpjs	✗	✗	✓
auerlia-path	✓	✓	✓

arbitrary properties to global object prototypes, potentially leading to subsequent inheritance by user-defined objects. Although detected by all analytical models (including the code model, commit message model, behavioral data model, and composed model), the commit was not associated with a CVE identifier.

7. Conclusion

In this study, we introduced a methodology for developing a multi-modal classifier capable of determining whether a given commit represents a security patch. The classifier comprises three sub-models: Developer Behavior Data, Commit Messages, and Code Patches, each trained on distinct data sources.

Various encoding techniques were employed to enhance the learning efficiency of the individual models. For instance, using the novel *Concat with comment* encoding method for Code patch, and using Conv1D model for classifying the developer behavioral data.

Upon training each sub-model, we evaluated the overall classification accuracy of the composed model. Notably, by unfreezing the sub-models' embedding layers during the final training phase, we achieved the highest classification accuracy of 94.52%.

Furthermore, we made efforts to gain insights into the factors that contribute to the model's improved learning performance and success in different scenarios. Our findings offer practical applications in the detection of unreported security patches, enabling quick warnings to the community for addressing potential security vulnerabilities.

In the future, we aim to improve our code representation module by incorporating the location of the changed code with respect to the unchanged code that surrounds it. Also, our training data is limited to security patches from GitHub that are referenced in CVE records; however, we plan to expand this by scraping security patches from CVE records that reference other version control platforms. Additionally, adversarial attacks, which may conceal security patches to evade detection, were not considered in our current analysis and could potentially bypass our detection mechanisms. Furthermore, we plan to evaluate the model's performance across different types of security vulnerabilities (e.g., buffer overflows, injection attacks) to identify whether detection accuracy varies based on the specific nature of the vulnerability.

CRediT authorship contribution statement

Nitzan Farhi: Writing – review & editing, Writing – original draft, Methodology, Investigation, Data curation. **Noam Koenigstein:** Writing – review & editing, Supervision, Resources, Methodology. **Yuval Shavitt:** Writing – review & editing, Writing – original draft, Supervision, Resources, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Failed detection examples

HHVM.[15] HHVM is a virtual machine for executing Hack programs. The reason the code model was not able to catch the bug is because the change was moving a snippet of data validation to the beginning of a function. The code model is not aware of location of changes in the code, and thus did not detect the security patch. The commit message do not contain any indicative information, and since it looks like a generic commit. The commits are done automatic by a bot, so the behavioral model did not detect anything special as well.

uftpd.[16] uftpd is a unix ftp server. The code model was not able to detect the security patch, because the change in the code was changing a usage of one variable to another, and since both variables already exist in the code, the change did not contain any information regarding those variables. The first variable describes a full path, and the second describes the relevant directory only. The commit message model was also not able to detect the security patch, as the commit message is short and does not contain any indication to a vulnerability, and lastly the behavioral model was not able to detect this security patch as well, as the commit was done as a part of version release.

Data availability

Code and data are supplied to this paper on submission.

References

[link]. URL https://cve.mitre.org/cve/.
Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G.S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., Zheng, X., 2015. TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org, URL https://www.tensorflow.org/.
Abuwala, H., Lim, J.Q., Do programming languages used in certain domains have an impact on bugs?.
Chacon, S., Straub, B., 2014. Pro git. Springer Nature.
Chen, Z., Fang, S., Monperrus, M., 2023. Supersonic: Learning to generate source code optimisations in c/c++. arXiv preprint arXiv:2309.14846.
Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y., 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. CoRR (arXiv:1406.1078).
Coelho, J., Valente, M.T., Silva, L.L., Shihab, E., 2018. Identifying unmaintained projects in github. In: The 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement. pp. 1–10.
Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.
Farhi, N., Koenigstein, N., Shavitt, Y., 2023. Detecting security patches via behavioral data in code repositories. arXiv preprint arXiv:2302.02112.
Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M., 2020. CodeBERT: A pre-trained model for programming and natural languages. CoRR abs/2002.08155, arXiv:2002.08155.
Hoang, T., Kang, H.J., Lo, D., Lawall, J., 2020. CC2Vec: Distributed representations of code changes. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering. pp. 518–529.
Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. Neural Comput. 9 (8), 1735–1780.
Horawalavithana, S., Bhattacharjee, A., Liu, R., Choudhury, N., Hall, L.O., Iamnitchi, A., 2019. Mentions of security vulnerabilities on reddit, twitter and github. In: IEEE/WIC/ACM International Conference on Web Intelligence. pp. 200–207.
Jung, T., 2021. CommitBERT: Commit message generation using pre-trained programming language model. CoRR abs/2105.14242, arXiv:2105.14242.
Kingma, D.P., Ba, J., 2014. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.

[15] https://github.com/facebook/hhvm/commit/190ffdf6c8b1ec443be202c7d69e63a7e3da25e3.
[16] https://github.com/troglobit/uftpd/commit/455b47d3756aed162d2d0ef7f40b549f3b5b30fe.

Le, T.H.M., Hin, D., Croft, R., Babar, M.A., 2021. DeepCVA: Automated commit-level vulnerability assessment with deep multi-task learning. In: 36th IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE, pp. 717–729.

Lee, J., Lim, P., Hooi, B., Divakaran, D.M., 2024. Multimodal large language models for phishing webpage detection and identification. arXiv preprint arXiv:2408.05941.

Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., Talwalkar, A., 2018. Hyperband: A novel bandit-based approach to hyperparameter optimization. J. Mach. Learn. Res. 18 (185), 1–52.

Li, F., Paxson, V., 2017. A large-scale empirical study of security patches. In: ACM SIGSAC Conference on Computer and Communications Security. pp. 2201–2215.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V., 2019. RoBERTa: A robustly optimized BERT pretraining approach. CoRR abs/1907.11692, arXiv:1907.11692.

Lundberg, S.M., Lee, S.-I., 2017. A unified approach to interpreting model predictions. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), Advances in Neural Information Processing Systems 30. Curran Associates, Inc., pp. 4765–4774, URL http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf.

Moraliyage, H., Sumanasena, V., De Silva, D., Nawaratne, R., Sun, L., Alahakoon, D., 2022. Multimodal classification of onion services for proactive cyber threat intelligence using explainable deep learning. IEEE Access 10, 56044–56056.

Nguyen, T.G., Le-Cong, T., Kang, H.J., Le, X.-B.D., Lo, D., 2022. VulCurator: a vulnerability-fixing commit detector. In: The 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 1726–1730.

Pan, K., Kim, S., Whitehead, E.J., 2009. Toward an understanding of bug fix patterns. Empir. Softw. Eng. 14, 286–315.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. PyTorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems. NeurIPS, pp. 8024–8035.

Pericherla, S., Egambaram, I., 2021. Cyberbullying detection on multi-modal data using pre-trained deep learning architectures. Ing. Solidar. 17 (3), 1–20.

Perl, H., Dechand, S., Smith, M., Arp, D., Yamaguchi, F., Rieck, K., Fahl, S., Acar, Y., 2015. VCCFinder: Finding potential vulnerabilities in open-source projects to assist code audits. In: 22nd ACM SIGSAC Conference on Computer and Communications Security. pp. 426–437.

Pickerill, P., Jungen, H.J., Ochodek, M., Maćkowiak, M., Staron, M., 2020. PHANTOM: Curating GitHub for engineered software projects using time-series clustering. Empir. Softw. Eng. 25 (4), 2897–2929.

Ratcliff, J.W., Metzener, D., 1988. Pattern matching: The gestalt approach. Dr. Dobb's J. 13 (7), 46.

Ray, B., Posnett, D., Filkov, V., Devanbu, P., 2014. A large scale study of programming languages and code quality in github. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. pp. 155–165.

Sabetta, A., Ponta, S.E., Lozoya, R.C., Bezzi, M., Sacchetti, T., Greco, M., Balogh, G., Hegedűs, P., Ferenc, R., Paramitha, R., et al., 2024. Known vulnerabilities of open source projects: Where are the fixes? IEEE Secur. Priv..

Steinwart, I., Christmann, A., 2008. Support Vector Machines. Springer Science & Business Media.

Sun, M., Wang, W., Feng, H., Sun, H., Zhang, Y., 2020. Identify vulnerability fix commits automatically using hierarchical attention network. EAI Endorsed Trans. Secur. Saf. 7 (23), e2.

Tan, X., Zhang, Y., Mi, C., Cao, J., Sun, K., Lin, Y., Yang, M., 2021. Locating the security patches for disclosed oss vulnerabilities with vulnerability-commit correlation ranking. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. pp. 3282–3299.

Tian, Y., Lawall, J., Lo, D., 2012. Identifying linux bug fixing patches. In: 2012 34th International Conference on Software Engineering. ICSE, pp. 386–396. http://dx.doi.org/10.1109/ICSE.2012.6227176.

Wang, X., Wang, S., Feng, P., Sun, K., Jajodia, S., 2021a. PatchDB: A large-scale security patch dataset. In: 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks. DSN, IEEE, pp. 149–160.

Wang, X., Wang, S., Feng, P., Sun, K., Jajodia, S., Benchaaboun, S., Geck, F., 2021b. PatchRNN: A deep learning-based system for security patch identification. In: IEEE Military Communications Conference. MILCOM, IEEE, pp. 595–600.

Zhou, Y., Siow, J.K., Wang, C., Liu, S., Liu, Y., 2021. SPI: Automated identification of security patches via commits. ACM Trans. Softw. Eng. Methodol. 31 (1), 1–27.

Zuo, F., Zhang, X., Song, Y., Rhee, J., Fu, J., 2023. Commit message can help: security patch detection in open source software via transformer. In: 2023 IEEE/ACIS 21st International Conference on Software Engineering Research, Management and Applications. SERA, IEEE, pp. 345–351.

Nitzan Farhi obtained his Bachelor of Science degree in Software Engineering from Ben Gurion University. He subsequently earned his Master of Science degree from the Department of Information Systems at the same institution. Currently, Nitzan is pursuing his Doctor of Philosophy degree in the Department of Electrical Engineering at Tel-Aviv University, focusing on the development of novel methods and applications for time series models.

Dr. Noam Koenigstein received the B.Sc. degree in computer science (cum laude) from the Technion Israel Institute of Technology, Haifa, Israel, in 2007 and an M.Sc. degree in electrical engineering from Tel-Aviv University, Tel-Aviv, Israel, in 2009. In 2013 he received a Ph.D. degree from the School of Electrical Engineering, Tel-Aviv University. In 2011, he joined the Xbox Machine Learning research team in Microsoft, where he developed the algorithm for Xbox recommendations serving millions of users world wide. Later he managed the recommendations research team for Microsoft's Store. In 2017 he joined Citi bank's Israeli Innovation Lab as Senior VP Head of Data Science overseeing all data science activities in the Israeli research center. In 2018 he joined the Industrial Engineering Department at Tel Aviv University as a faculty member. Today, he heads the DELTA Lab for applied deep-learning technologies at the Department for Industrial Engineering at Tel-Aviv University where students are working on applying machine learning algorithms to a diverse set of real-world problems.

Prof. Yuval Shavitt is Professor of Electrical Engineering at Tel Aviv University. Before joining Tel Aviv University, he worked for four years at the Networking Center of Bell Labs, Holmdel, NJ. He has published seminal papers in the fields of caching, routing, IP hijack attacks, and network measurements. In 2004, he founded the DIMES project for mapping the Internet infrastructure using thousands of lightweight software agents, which revolutionized the field of Internet measurement and mapping. Data gathered by DIMES was used by academicians worldwide. In 2014 he established BGProtect, a company that uses the DIMES approach to protect nations and large organizations against IP hijack attacks and provide network infrastructure threat intelligence.